BÀI 1/6 — EXCEPTION TRONG ỨNG DỤNG C# THỰC TẾ

Tài liệu ôn tập trước khi học Backend .NET • Bao gồm lý thuyết, ví dụ, bài tập thực hành, đáp án, giải thích lỗi và câu hỏi tự kiểm tra.

Đối tượng: Người học đã biết OOP, Collections, LINQ và try/catch cơ bản. **Mục tiêu:** lựa chọn loại Exception phù hợp, hiểu cách lỗi lan qua các tầng, ném lại lỗi đúng và bọc lỗi bằng InnerException.

MỤC LỤC

- 1. Tổng quan về Exception và luồng thực thi
- 2. Các loại Exception thường gặp
- 3. Exception Propagation qua các phương thức
- 4. throw, throw ex, throw new và InnerException
- 5. Khi nào nên catch? Liên hệ với ASP.NET Core
- 6. Đề bài thực hành (3 bài)
- 7. Đáp án tham khảo đầy đủ
- 8. Phân tích những lỗi dễ mắc trong bài nộp
- 9. Câu hỏi lý thuyết và đáp án
- 10. Checklist trước khi sang Nullable Reference Types

---

1. Tổng quan về Exception và luồng thực thi

Exception là đối tượng mô tả lỗi hoặc một tình huống bất thường khi chương trình đang thực thi. Khi gặp **throw**, luồng chạy bình thường bị gián đoạn; hệ thống tìm khối **catch** phù hợp ở vị trí hiện tại hoặc các phương thức gọi phía trên.

Từ khóa	Ý nghĩa
try	Đặt các lệnh có thể phát sinh Exception.
catch	Nhận và xử lý Exception khớp kiểu.
finally	Thường chạy khi rời try/catch, phù hợp để dọn dẹp tài nguyên.
throw	Ném Exception mới hoặc ném lại Exception hiện tại.

```
try
{
    account.Withdraw(-50);
    Console.WriteLine("Withdrawal completed.");
}
catch (ArgumentOutOfRangeException ex)
{
    Console.WriteLine($"Error: {ex.Message}");
}
Console.WriteLine("Program continues.");
```

Khi Withdraw(-50) ném lỗi, câu lệnh "Withdrawal completed." không chạy. Nếu catch xử lý được lỗi thì "Program continues." vẫn chạy.

2. Những loại Exception cần ghi nhớ

Loại	Ví dụ / nguyên tắc
ArgumentException	ID rỗng, toàn khoảng trắng hoặc tham số không hợp lệ.
ArgumentNullException	Đối tượng bắt buộc nhưng đầu vào là null.
ArgumentOutOfRangeException	Giá trị ngoài miền cho phép: amount <= 0, balance âm.
InvalidOperationException	Thao tác không phù hợp với trạng thái hiện tại: số dư không đủ.
KeyNotFoundException	Truy cập khóa không tồn tại trong Dictionary bằng chỉ mục.
FormatException	int.Parse("abc") không thể phân tích chuỗi thành số.

Phân biệt cần nhớ: số tiền rút <= 0 là tham số không hợp lệ; số tiền rút dương nhưng lớn hơn số dư là điều kiện nghiệp vụ không đáp ứng. Trong bài tập này, trường hợp thứ hai dùng InvalidOperationException.

3. Exception Propagation: Lỗi lan lên tầng gọi

Giả sử Main() gọi BankService.ProcessWithdrawal(), rồi BankService gọi BankAccount.Withdraw(). Nếu Withdraw() ném lỗi và không bắt, lỗi đi ngược về BankService. Nếu BankService ném lại, lỗi tiếp tục đi về Main().

```
// BankAccount
public void Withdraw(decimal amount)
{
    if (amount > Balance)
        throw new InvalidOperationException("Insufficient funds.");
    Balance -= amount;
}

// BankService
public void ProcessWithdrawal(BankAccount account, decimal amount)
{
    try { account.Withdraw(amount); }
    catch (InvalidOperationException)
    {
        Console.WriteLine("Withdrawal processing failed.");
        throw;
    }
}
```

Luồng: Main → ProcessWithdrawal → Withdraw → InvalidOperationException → catch tại BankService → throw; → catch tại Main. Khối catch của BankService không làm Exception "biến mất" nếu tiếp tục dùng throw;.

4. throw;, throw ex; và throw new

Cú pháp	Ý nghĩa	Khuyến nghị
throw;	Ném lại Exception đang bắt, giữ thông tin lỗi gốc trong stack trace.	Dùng khi cần ghi log rồi chuyển lỗi lên.
throw ex;	Ném lại đối tượng nhưng làm đặt lại/thay đổi stack trace ở vị trí ném lại.	Tránh dùng để rethrow.
throw new ...	Tạo một Exception mới. Có thể truyền lỗi gốc vào InnerException.	Dùng khi tạo lỗi hoặc bổ sung ngữ cảnh.

4.1. Rethrow đúng

```
catch (InvalidOperationException)
{
    Console.WriteLine("Withdrawal processing failed.");
    throw; // Giữ lại vị trí l□i g□c
}
```

4.2. Bọc Exception bằng InnerException

```
public static int ConvertAge(string input)
{
    try
    {
        return int.Parse(input);
    }
    catch (FormatException ex)
    {
        throw new InvalidOperationException("Cannot convert age.", ex);
    }
}
```

ex là gì? Trong catch (FormatException ex), ex là biến tham chiếu đến đối tượng FormatException đã được ném từ int.Parse(). Khi truyền ex vào constructor InvalidOperationException(message, ex), Exception mới giữ Exception cũ trong thuộc tính **InnerException**.

```
try
{
    int age = ConvertAge("abc");
}
catch (InvalidOperationException ex)
{
    Console.WriteLine(ex.Message);
    Console.WriteLine(ex.InnerException?.Message);
}
```

Thông báo ngoài: "Cannot convert age.". Thông báo trong: lỗi định dạng gốc của int.Parse("abc"). Ký hiệu **?.** giúp tránh truy cập thuộc tính nếu InnerException là null.

4.3. Lưu ý về code không thể chạy

```
catch (FormatException ex)
{
    throw new InvalidOperationException("Cannot convert age.", ex);
    throw; // Không th đ n dòng này: mã unreachable
}
```

Ngay khi throw new thực thi, điều khiển rời khối catch. Chỉ giữ một lệnh throw phù hợp, không đặt throw; sau throw new.

5. Khi nào nên catch? Liên hệ Backend .NET

- Validate tham số ngay tại điểm nhận dữ liệu; ném Exception rõ ràng nếu dữ liệu vô nghĩa.
- Service nghiệp vụ thường tự bảo vệ quy tắc (ví dụ không cho rút quá số dư).
- Không bắt Exception chỉ để che lỗi hoặc in thông báo trong mọi phương thức.
- Có thể catch để khôi phục trạng thái, ghi log, hoặc thêm ngữ cảnh; nếu chưa xử lý xong thì ném tiếp.
- Khi xây dựng ASP.NET Core, có thể xử lý lỗi tập trung ở tầng HTTP (middleware/exception handler) để trả response phù hợp. Exception C# không tự động tương ứng với mã HTTP.

Gợi ý: không dùng exception cho mọi nhánh nghiệp vụ thường gặp; về sau, tùy thiết kế API có thể dùng kết quả nghiệp vụ (Result) hoặc exception được chuẩn hóa.

6. ĐỀ BÀI THỰC HÀNH

Bài 1 – BankAccount: Validation và Withdraw

Tạo BankAccount với AccountNumber (string, chỉ đọc) và Balance (decimal, private set). Constructor kiểm tra mã tài khoản không trắng, số dư ban đầu >= 0; chuẩn hóa AccountNumber bằng Trim(). Viết Withdraw(amount): amount <= 0 → ArgumentOutOfRangeException; amount > Balance → InvalidOperationException; thành công → giảm Balance. Các tình huống lỗi không được làm thay đổi Balance.

Kiểm thử	Kết quả kỳ vọng
Balance=1000, Withdraw(200)	Balance còn 800.
Sau đó Withdraw(-50)	ArgumentOutOfRangeException, Balance vẫn 800.
Sau đó Withdraw(1000)	InvalidOperationException, Balance vẫn 800.
Constructor với balance=-500	ArgumentOutOfRangeException.
Withdraw(800) khi Balance=800	Thành công, Balance=0.

Bài 2 – BankService: Exception Propagation

Viết ProcessWithdrawal(BankAccount account, decimal amount). Nếu account null: ArgumentNullException. Gọi account.Withdraw(amount). Nếu bắt được InvalidOperationException: in "Withdrawal processing failed." rồi dùng throw;. Trong Main, gọi rút vượt số dư và bắt InvalidOperationException để in thông báo gốc.

Bài 3 – ConvertAge: Exception Wrapping

Viết static int ConvertAge(string input). Gọi int.Parse(input); nếu FormatException thì ném InvalidOperationException mới với Message "Cannot convert age." và InnerException là lỗi ban đầu. Trong Main, thử ConvertAge("abc") và in cả Message lẫn InnerException?.Message.

7. ĐÁP ÁN THAM KHẢO

Ba file dưới đây là một đáp án C# Console hoàn chỉnh (sử dụng implicit usings hoặc thêm using System; nếu dự án chưa cấu hình).

7.1. BankAccount.cs

```
public class BankAccount
{
    public string AccountNumber { get; }
    public decimal Balance { get; private set; }

    public BankAccount(string accountNumber, decimal balance)
    {
        if (string.IsNullOrWhiteSpace(accountNumber))
            throw new ArgumentException(
                "Account number cannot be blank.", nameof(accountNumber));
        if (balance < 0)
            throw new ArgumentOutOfRangeException(
                nameof(balance), "Balance cannot be negative.");

        AccountNumber = accountNumber.Trim();
        Balance = balance;
    }

    public void Withdraw(decimal amount)
    {
        if (amount <= 0)
            throw new ArgumentOutOfRangeException(
                nameof(amount), "Amount must be greater than zero.");
        if (amount > Balance)
            throw new InvalidOperationException("Insufficient funds.");

        Balance -= amount;
    }
}
```

Giải thích: Balance có private set nên mã ngoài class chỉ đọc. Validation kiểm tra tham số balance, không kiểm tra thuộc tính Balance chưa gán. Withdraw chỉ cập nhật sau khi vượt qua các kiểm tra.

7.2. BankService.cs

```
public class BankService
{
    public void ProcessWithdrawal(BankAccount account, decimal amount)
    {
        if (account is null)
            throw new ArgumentNullException(nameof(account));

        try
        {
            account.Withdraw(amount);
        }
        catch (InvalidOperationException)
        {
            Console.WriteLine("Withdrawal processing failed.");
            throw;
        }
    }
}
```

Giải thích: catch (InvalidOperationException) khớp lỗi "Insufficient funds." từ Withdraw. throw; tiếp tục gửi cùng lỗi lên Main. Lỗi amount <= 0 không bị catch này bắt mà tự lan lên trên.

7.3. Program.cs – Chạy đủ các tình huống

```
public class Program
{
    public static int ConvertAge(string input)
    {
        try
        {
            return int.Parse(input);
        }
        catch (FormatException ex)
        {
            throw new InvalidOperationException("Cannot convert age.", ex);
        }
    }

    public static void Main(string[] args)
    {
        var account = new BankAccount(" 1233 ", 1000);
        account.Withdraw(200);
        Console.WriteLine($"After valid withdrawal: {account.Balance}");

        try { account.Withdraw(-50); }
        catch (ArgumentOutOfRangeException ex)
        {
            Console.WriteLine($"Invalid amount: {ex.Message}");
        }
        Console.WriteLine($"After failed withdrawal: {account.Balance}");

        try { account.Withdraw(1000); }
        catch (InvalidOperationException ex)
        {
            Console.WriteLine($"Insufficient funds: {ex.Message}");
        }
        Console.WriteLine($"Balance unchanged: {account.Balance}");

        try { _ = new BankAccount("X01", -500); }
        catch (ArgumentOutOfRangeException ex)
        {
            Console.WriteLine($"Invalid initial balance: {ex.Message}");
        }

        var service = new BankService();
        try { service.ProcessWithdrawal(account, 1000); }
        catch (InvalidOperationException ex)
        {
            Console.WriteLine($"Main caught: {ex.Message}");
        }

        try { Console.WriteLine($"Age: {ConvertAge("abc")}"); }
        catch (InvalidOperationException ex)
        {
            Console.WriteLine($"Conversion error: {ex.Message}");
            Console.WriteLine($"Original error: {ex.InnerException?.Message}");
        }

        account.Withdraw(800);
        Console.WriteLine($"After exact withdrawal: {account.Balance}");
    }
}
```

Ghi chú: chương trình trên đặt trường hợp rút đúng 800 ở cuối, vì sau lần rút 200 thành công, số dư là 800; các lần rút thất bại không làm đổi số dư. Với tài khoản khác, bạn cũng có thể tổ chức các test độc lập.

7.4. Kết quả kỳ vọng (rút gọn)

```
After valid withdrawal: 800
Invalid amount: ...
After failed withdrawal: 800
Insufficient funds: Insufficient funds.
Balance unchanged: 800
Invalid initial balance: ...
Withdrawal processing failed.
Main caught: Insufficient funds.
Conversion error: Cannot convert age.
Original error: The input string 'abc' was not in a correct format.
After exact withdrawal: 0
```

Các dòng ArgumentOutOfRangeException và thông báo FormatException có thể kèm tên tham số hoặc cách diễn đạt hơi khác tùy phiên bản .NET.

8. Phân tích các lỗi thực tế trong bài đã nộp

Lỗi	Tại sao sai	Cách sửa
if (Balance < 0) trong constructor	Balance chưa được gán tham số, mặc định là 0.	Kiểm tra if (balance < 0).
Không đủ tiền → ArgumentOutOfRangeException	Không đúng quy ước của bài và không khớp catch trong BankService.	Ném InvalidOperationException.
Main chỉ catch ArgumentOutOfRangeException khi gọi BankService	Sau khi sửa Withdraw, lỗi không đủ tiền là InvalidOperationException.	Sửa catch tương ứng.
throw new ...; throw;	Dòng thứ hai không bao giờ chạy.	Xóa throw; thừa.
Chỉ in ex.Message sau khi wrap	Chưa quan sát nguyên nhân kỹ thuật gốc.	In ex.InnerException?.Message.

9. Câu hỏi ôn tập VÀ ĐÁP ÁN

1. Khi một phương thức ném lỗi nhưng không có catch thì sao?

Đáp án: Exception lan ngược lên các phương thức gọi. Nếu không có catch phù hợp ở bất kỳ tầng nào, nó trở thành lỗi không được xử lý.

2. throw; khác throw ex; thế nào?

Đáp án: throw; ném lại Exception đang bắt mà giữ tốt thông tin vị trí lỗi gốc; throw ex; đặt lại/thay đổi stack trace tại vị trí ném lại, nên không dùng để rethrow thông thường.

3. Vì sao BankService bắt rồi Main vẫn bắt được?

Đáp án: Vì sau khi BankService bắt InvalidOperationException, nó dùng throw; để ném lại. Exception được truyền lên Main.

4. Nếu BankService chỉ catch InvalidOperationException còn Withdraw ném ArgumentOutOfRangeException?

Đáp án: catch đó không khớp nên bị bỏ qua; lỗi tiếp tục lan lên Main hoặc tầng gọi cao hơn, chứ không bị mất.

5. Khác nhau giữa tham số rút âm và rút vượt số dư?

Đáp án: Rút âm vi phạm quy tắc của tham số nên dùng ArgumentOutOfRangeException. Rút vượt số dư là điều kiện nghiệp vụ không đáp ứng nên bài tập dùng InvalidOperationException.

6. Trong catch (FormatException ex), ex là gì?

Đáp án: ex là biến tham chiếu đến đối tượng FormatException ban đầu, có các thông tin như Message, StackTrace.

7. InnerException dùng để làm gì?

Đáp án: Giữ nguyên nhân lỗi gốc khi tạo một Exception mới có thông điệp/ngữ cảnh rõ hơn.

8. Có cần catch trong mọi phương thức không?

Đáp án: Không. Chỉ catch khi có xử lý có ý nghĩa (khôi phục, ghi log, bổ sung ngữ cảnh). Nếu không xử lý được, để lỗi lan lên.

9. Dòng sau throw trong cùng nhánh có được chạy không?

Đáp án: Không. throw làm luồng thực thi rời nhánh hiện tại; các dòng tiếp sau trong nhánh đó không thể chạy.

10. `ex.InnerException?.Message` có ý nghĩa gì?

Đáp án: Lấy thông báo của lỗi bên trong nếu InnerException không null; nếu null thì biểu thức cho giá trị null.

11. Exception khác HTTP status code thế nào?

Đáp án: Exception thuộc luồng xử lý C#; HTTP status code thuộc phản hồi giao thức web. Trong ASP.NET Core cần cơ chế ánh xạ phù hợp giữa lỗi và response.

10. Checklist trước khi học Bài 2/6 – Nullable Reference Types

- Phân biệt được ArgumentException, ArgumentNullException, ArgumentOutOfRangeException, InvalidOperationException và FormatException.
- Giải thích được luồng Main → BankService → BankAccount và chiều lan của Exception.
- Biết rethrow bằng throw; và không đặt throw; sau throw new.
- Bọc được một FormatException bằng InvalidOperationException có InnerException.
- Dùng try/catch riêng để kiểm thử các trường hợp lỗi và kiểm tra trạng thái dữ liệu không đổi.

Tiếp theo: Bài 2/6 – Nullable Reference Types: Product?, string?, kiểm tra null, toán tử ?. và ??; giúp viết code C# an toàn hơn khi phát triển ASP.NET Core.